

Contactless Fingerprint Quality Assessment

Project Overview

The **Contactless Fingerprint Quality Assessment** system is a Python-based application developed to evaluate the quality of contactless fingerprint images. The application analyzes fingerprint images and calculates quality metrics that help determine whether an image is suitable for biometric recognition.

The project provides a simple graphical web interface built with **Streamlit**, allowing users to upload fingerprint images and instantly receive quality scores and feedback.

Objectives

- Assess the quality of contactless fingerprint images.
 - Detect image characteristics that affect fingerprint recognition.
 - Provide an easy-to-use interface for quality evaluation.
 - Help improve the reliability of fingerprint authentication systems.
-

Features

- Upload fingerprint images.
 - Automatic fingerprint quality analysis.
 - Displays quality score.
 - User-friendly Streamlit interface.
 - Lightweight and easy to run locally.
 - Supports common image formats such as JPG, JPEG, and PNG.
-

Technologies Used

Technology	Purpose
Python	Programming Language
Streamlit	Web Application Framework
OpenCV	Image Processing
NumPy	Numerical Computations
Pillow (PIL)	Image Handling

Project Structure

```
contactless-fingerprint-qc/
├── quality_app.py           # Streamlit application
├── quality_assessment.py    # Quality assessment logic
├── requirements.txt         # Project dependencies
├── .gitignore
├── test_dataset/
│   ├── Blur/
│   ├── Good/
│   ├── Dry/
│   ├── Wet/
│   └── ...
└── README.md
```

System Requirements

- Python 3.10 or later
 - pip
 - Streamlit
 - OpenCV
 - NumPy
 - Pillow
-

Installation

Clone the Repository

```
git clone
https://github.com/ruheenatasneem/Contactless_FingerPrint_Quality.git
```

Move into the Project Folder

```
cd Contactless_FingerPrint_Quality
```

Install Dependencies

```
pip install -r requirements.txt
```

Running the Application

Start the Streamlit application using:

```
streamlit run quality_app.py
```

The application will automatically open in your web browser.

Working Procedure

1. Launch the Streamlit application.
 2. Upload a fingerprint image.
 3. The image is processed using OpenCV.
 4. Image quality metrics are calculated.
 5. The overall fingerprint quality score is displayed.
 6. Users can determine whether the fingerprint image is suitable for biometric applications.
-

Quality Assessment Process

The application evaluates the uploaded fingerprint image using image-processing techniques such as:

- Image loading
- Grayscale conversion (if required)
- Contrast analysis
- Sharpness evaluation
- Blur detection
- Brightness estimation
- Overall quality score calculation

These measurements help determine whether the fingerprint image is of acceptable quality.

Advantages

- Fast quality assessment.
 - Easy to use.
 - Lightweight application.
 - Can be extended for machine learning models.
 - Useful for biometric preprocessing.
 - Open-source and customizable.
-

Limitations

- Works only with supported image formats.
 - Quality depends on the uploaded image.
 - Does not perform fingerprint matching.
 - Does not include liveness detection.
-

Future Enhancements

- Deep learning-based quality assessment.
 - Fingerprint liveness detection.
 - Automatic segmentation.
 - Ridge clarity visualization.
 - Batch processing of multiple images.
 - PDF report generation.
 - Cloud deployment.
-

Applications

- Biometric Authentication
 - Attendance Systems
 - Mobile Identity Verification
 - Banking Security
 - e-Governance
 - Access Control Systems
 - Research in Biometrics
-

Conclusion

The **Contactless Fingerprint Quality Assessment** project provides an efficient solution for evaluating the quality of contactless fingerprint images before they are used for biometric recognition. By identifying poor-quality images early, the system can improve the accuracy and reliability of fingerprint-based authentication systems. The project demonstrates practical use of Python, Streamlit, and OpenCV for image quality analysis and provides a strong foundation for future enhancements in biometric technology.

Code:

=====

Contactless Fingerprint Quality Assessment
quality_assessment.py

Author : Ruheena Tasneem
Description:
Core Quality Assessment Module

This module performs

1. Blur Detection
2. Brightness Detection
3. Glare Detection
4. ROI Completeness
5. Ridge Clarity
6. Composite Score

7. Quality Gate

```
=====
"""

import cv2
import numpy as np

# =====
# Blur Detection
# =====

def check_blur(image_bgr, threshold=10.0):
    """
    Detects whether image is blurry using
    Laplacian Variance.
    """

    gray = cv2.cvtColor(image_bgr, cv2.COLOR_BGR2GRAY)

    blur_score = cv2.Laplacian(
        gray,
        cv2.CV_64F
    ).var()

    result = {
        "blur_score": round(float(blur_score), 2),
        "is_blurry": blur_score < threshold
    }

    return result

# =====
# Brightness Detection
# =====

def check_brightness(
    image_bgr,
    min_thresh=50,
    max_thresh=210
):

    gray = cv2.cvtColor(
        image_bgr,
        cv2.COLOR_BGR2GRAY
    )

    brightness = np.mean(gray)

    result = {
        "brightness": round(float(brightness), 2),
        "too_dark": brightness < min_thresh,
        "too_bright": brightness > max_thresh
    }

    return result

# =====
# Glare Detection
# =====

def check_glare(
    image_bgr,
    max_glare_ratio=0.05
):

    gray = cv2.cvtColor(
        image_bgr,
```

```

        cv2.COLOR_BGR2GRAY
    )

    glare_pixels = np.sum(gray > 240)

    total_pixels = gray.size

    glare_fraction = glare_pixels / total_pixels

    result = {
        "has_glare": glare_fraction > max_glare_ratio,
        "glare_fraction": round(float(glare_fraction), 4)
    }

    return result

# =====
# ROI Completeness
# =====

def check_roi_completeness(
    image_bgr,
    min_roi_ratio=0.15
):

    gray = cv2.cvtColor(
        image_bgr,
        cv2.COLOR_BGR2GRAY
    )

    blur = cv2.GaussianBlur(
        gray,
        (5, 5),
        0
    )

    _ , thresh = cv2.threshold(
        blur,
        0,
        255,
        cv2.THRESH_BINARY + cv2.THRESH_OTSU
    )

    foreground_pixels = np.sum(thresh > 0)

    total_pixels = gray.size

    roi_fraction = foreground_pixels / total_pixels

    result = {
        "roi_fraction": round(float(roi_fraction), 4),
        "roi_complete": roi_fraction >= min_roi_ratio
    }

    return result

# =====
# Ridge Clarity
# =====

def check_ridge_clarity(
    image_bgr,
    threshold=15.0
):

    gray = cv2.cvtColor(
        image_bgr,
        cv2.COLOR_BGR2GRAY
    )

```

```

kernel = cv2.getGaborKernel(
    (21, 21),
    sigma=5,
    theta=np.pi / 4,
    lambd=10,
    gamma=0.5,
    psi=0
)

filtered = cv2.filter2D(
    gray,
    cv2.CV_64F,
    kernel
)

ridge_score = np.var(filtered) / 100

result = {
    "ridge_score": round(float(ridge_score), 2),
    "ridges_clear": ridge_score >= threshold
}

return result

# =====
# Normalize Blur
# =====

def normalize_blur(score):

    return min(1.0, score / 50.0)

# =====
# Normalize Brightness
# =====

def normalize_brightness(score):

    value = 1.0 - abs(score - 128) / 128

    return max(0.0, value)

# =====
# Normalize Glare
# =====

def normalize_glare(glare):

    value = 1.0 - glare / 0.05

    return max(0.0, value)

# =====
# Normalize ROI
# =====

def normalize_roi(roi):

    return min(1.0, roi / 0.35)

# =====
# Normalize Ridge
# =====

def normalize_ridge(score):

```

```

return min(1.0, score / 30.0)

# =====
# Composite Score
# =====

def calculate_composite_score(
    blur_result,
    bright_result,
    glare_result,
    roi_result,
    ridge_result
):

    n_blur = normalize_blur(
        blur_result["blur_score"]
    )

    n_brightness = normalize_brightness(
        bright_result["brightness"]
    )

    n_glare = normalize_glare(
        glare_result["glare_fraction"]
    )

    n_roi = normalize_roi(
        roi_result["roi_fraction"]
    )

    n_ridge = normalize_ridge(
        ridge_result["ridge_score"]
    )

    score = (
        0.25 * n_blur +
        0.15 * n_brightness +
        0.15 * n_glare +
        0.20 * n_roi +
        0.25 * n_ridge
    ) * 100

    return round(score, 2)

# =====
# Guidance Message
# =====

def get_guidance(
    blur_result,
    bright_result,
    glare_result,
    roi_result,
    ridge_result
):
    """
    Returns guidance message based on failed quality check.
    """

    if blur_result["is_blurry"]:
        return "Too blurry - Hold your phone steady and refocus."

    if bright_result["too_dark"]:
        return "Too dark - Move to a brighter area or turn on flash."

    if bright_result["too_bright"]:
        return "Too bright - Reduce direct lighting."

    if glare_result["has_glare"]:
        return "Glare detected - Tilt your finger slightly."

```



```

if not roi_result["roi_complete"]:
    return "Finger not fully visible - Move finger closer."

if not ridge_result["ridges_clear"]:
    return "Fingerprint ridges unclear - Clean the camera lens."

return "Good capture - Ready for processing."

```

```

# =====
# Master Quality Gate
# =====

```

```

def quality_gate(image_path_or_array):
    """
    Main function for evaluating fingerprint quality.
    """

    # Read Image
    if isinstance(image_path_or_array, str):
        img = cv2.imread(image_path_or_array)
    else:
        img = image_path_or_array

    if img is None:
        raise ValueError("Image could not be loaded.")

    # Run all quality checks
    blur_result = check_blur(img)

    bright_result = check_brightness(img)

    glare_result = check_glare(img)

    roi_result = check_roi_completeness(img)

    ridge_result = check_ridge_clarity(img)

    # Calculate Composite Score
    composite_score = calculate_composite_score(
        blur_result,
        bright_result,
        glare_result,
        roi_result,
        ridge_result
    )

    # Hard Failure Conditions
    hard_failure = (
        blur_result["is_blurry"] or
        bright_result["too_dark"] or
        bright_result["too_bright"] or
        glare_result["has_glare"] or
        not roi_result["roi_complete"] or
        not ridge_result["ridges_clear"]
    )

    passed = composite_score >= 60 and not hard_failure

    guidance = get_guidance(
        blur_result,
        bright_result,
        glare_result,
        roi_result,
        ridge_result
    )

    result = {
        "passed": passed,
        "composite_score": composite_score,

```

```

        "blur": blur_result,
        "brightness": bright_result,
        "glare": glare_result,
        "roi": roi_result,
        "ridge": ridge_result,
        "guidance": guidance
    }

    return result

# =====
# Print Results
# =====

def print_result(result):
    """
    Prints results in a readable format.
    """

    print("\n")
    print("=" * 60)
    print("CONTACTLESS FINGERPRINT QUALITY REPORT")
    print("=" * 60)

    print(f"Composite Score : {result['composite_score']} / 100")

    if result["passed"]:
        print("Overall Result : PASS")
    else:
        print("Overall Result : FAIL")

    print("-" * 60)

    print("Blur Score      :", result["blur"]["blur_score"])
    print("Brightness      :", result["brightness"]["brightness"])
    print("Glare Fraction    :", result["glare"]["glare_fraction"])
    print("ROI Fraction      :", result["roi"]["roi_fraction"])
    print("Ridge Score       :", result["ridge"]["ridge_score"])

    print("-" * 60)

    print("Guidance:")
    print(result["guidance"])

    print("=" * 60)

# =====
# Main Function
# =====

if __name__ == "__main__":

    image_path = input("Enter image path : ")

    try:

        result = quality_gate(image_path)

        print_result(result)

    except Exception as e:

        print("Error :", e)
    """

```

contactless-fingerprint-qc/
 quality_app.py
 Contactless Fingerprint Quality Assessment
 quality_app.py

Author : Ruheena Tasneem

Streamlit User Interface

```
=====
"""

import streamlit as st
import cv2
import numpy as np
from PIL import Image

from quality_assessment import quality_gate

# -----
# Page Configuration
# -----

st.set_page_config(
    page_title="Fingerprint Quality Assessment",
    page_icon="👉",
    layout="wide"
)

# -----
# Title
# -----

st.title("👉 Contactless Fingerprint Quality Assessment")
st.write("Upload a contactless fingerprint image to evaluate its quality.")

# -----
# Sidebar
# -----

st.sidebar.title("Quality Thresholds")

blur_threshold = st.sidebar.slider(
    "Blur Threshold",
    min_value=5.0,
    max_value=50.0,
    value=10.0,
    step=1.0
)

dark_threshold = st.sidebar.slider(
    "Minimum Brightness",
    min_value=20,
    max_value=100,
    value=50
)

bright_threshold = st.sidebar.slider(
    "Maximum Brightness",
    min_value=150,
    max_value=255,
    value=210
)

glare_threshold = st.sidebar.slider(
    "Maximum Glare Ratio",
    min_value=0.01,
    max_value=0.15,
    value=0.05,
    step=0.01
)

st.sidebar.markdown("---")
st.sidebar.write("Developed using")
st.sidebar.write("• Python")
st.sidebar.write("• OpenCV")
```

```

st.sidebar.write("• NumPy")
st.sidebar.write("• Streamlit")

# -----
# File Upload
# -----

uploaded_file = st.file_uploader(
    "Choose Fingerprint Image",
    type=["jpg", "jpeg", "png"]
)

# -----
# Main Section
# -----

if uploaded_file is not None:

    image = Image.open(uploaded_file)

    image = image.convert("RGB")

    image_np = np.array(image)

    image_bgr = cv2.cvtColor(
        image_np,
        cv2.COLOR_RGB2BGR
    )

    result = quality_gate(image_bgr)

    col1, col2 = st.columns(2)

    # -----
    # Left Column
    # -----

    with col1:

        st.subheader("Uploaded Image")

        st.image(
            image,
            use_container_width=True
        )

    # -----
    # Right Column
    # -----

    with col2:

        st.subheader("Quality Assessment")

        score = result["composite_score"]

        st.metric(
            label="Composite Score",
            value=f"{score}/100"
        )

        if result["passed"]:

            st.success("✔ PASS")

        else:

            st.error("✗ FAIL")

        st.info(result["guidance"])

```

```

        st.progress(min(score / 100, 1.0))

st.markdown("---")

# -----
# Metric Cards
# -----

st.subheader("Quality Metrics")

c1, c2, c3 = st.columns(3)

with c1:

    st.write("### Blur")

    if result["blur"]["is_blurry"]:

        st.error("FAIL")

    else:

        st.success("PASS")

    st.write(
        f"Score : {result['blur']['blur_score']}"
    )

with c2:

    st.write("### Brightness")

    if (
        result["brightness"]["too_dark"]
        or
        result["brightness"]["too_bright"]
    ):

        st.error("FAIL")

    else:

        st.success("PASS")

    st.write(
        f"Brightness : {result['brightness']['brightness']}"
    )

with c3:

    st.write("### Glare")

    if result["glare"]["has_glare"]:

        st.error("FAIL")

    else:

        st.success("PASS")

    st.write(
        f"Ratio : {result['glare']['glare_fraction']}"
    )

st.markdown("---")

c4, c5 = st.columns(2)

with c4:

    st.write("### ROI Completeness")

```

```

if result["roi"]["roi_complete"]:

    st.success("PASS")

else:

    st.error("FAIL")

st.write(
    f"ROI : {result['roi']['roi_fraction']}"
)

with c5:

    st.write("### Ridge Clarity")

    if result["ridge"]["ridges_clear"]:

        st.success("PASS")

    else:

        st.error("FAIL")

    st.write(
        f"Score : {result['ridge']['ridge_score']}"
    )

st.markdown("---")

# -----
# Summary Table
# -----

st.subheader("Summary")

summary = {
    "Metric": [
        "Blur",
        "Brightness",
        "Glare",
        "ROI",
        "Ridge"
    ],
    "Status": [
        "FAIL" if result["blur"]["is_blurry"] else "PASS",

        "FAIL"
        if (
            result["brightness"]["too_dark"]
            or
            result["brightness"]["too_bright"]
        )
        else "PASS",

        "FAIL"
        if result["glare"]["has_glare"]
        else "PASS",

        "PASS"
        if result["roi"]["roi_complete"]
        else "FAIL",

        "PASS"
        if result["ridge"]["ridges_clear"]
        else "FAIL"
    ]
}

```

```

st.table(summary)

st.markdown("----")

st.subheader("Guidance")

st.success(result["guidance"])

# -----
# Footer
# -----

else:

    st.info("Please upload a fingerprint image.")

st.markdown("----")

st.caption(
    "Contactless Fingerprint Quality Assessment using OpenCV and Streamlit"
)

"""
=====
Contactless Fingerprint Quality Assessment
test_quality.py

Author : Ruheena Tasneem

Batch Testing Script

This program:

1. Reads all images from test_dataset
2. Runs quality assessment
3. Displays results
4. Saves results to CSV
=====
"""

import os
import glob
import pandas as pd

from quality_assessment import quality_gate

# =====
# Get Image List
# =====

def get_image_list(dataset_folder="test_dataset"):
    """
    Returns all image paths.
    """

    image_paths = []

    extensions = [
        "*.jpg",
        "*.jpeg",
        "*.png"
    ]

    categories = [
        "good",
        "blurry",
        "dark",
        "glare"
    ]

```

```
for category in categories:
```

```
    folder = os.path.join(
        dataset_folder,
        category
    )
```

```
for ext in extensions:
```

```
    files = glob.glob(
        os.path.join(folder, ext)
    )
```

```
    image_paths.extend(files)
```

```
return image_paths
```

```
# =====
# Evaluate Dataset
# =====
```

```
def evaluate_dataset(dataset_folder="test_dataset"):
```

```
    image_paths = get_image_list(dataset_folder)
```

```
    if len(image_paths) == 0:
```

```
        print("No images found.")
```

```
    return
```

```
    records = []
```

```
    print("\n")
    print("=" * 90)
    print("CONTACTLESS FINGERPRINT QUALITY TEST")
    print("=" * 90)
```

```
    for image_path in image_paths:
```

```
        folder_name = os.path.basename(
            os.path.dirname(image_path)
        )
```

```
        file_name = os.path.basename(
            image_path
        )
```

```
    try:
```

```
        result = quality_gate(image_path)
```

```
        records.append({
```

```
            "Filename":
                file_name,
```

```
            "Category":
                folder_name,
```

```
            "Passed":
                result["passed"],
```

```
            "Composite Score":
                result["composite_score"],
```

```
            "Blur Score":
                result["blur"]["blur_score"],
```



```

        "Brightness":
            result["brightness"]["brightness"],

        "Glare":
            result["glare"]["glare_fraction"],

        "ROI":
            result["roi"]["roi_fraction"],

        "Ridge":
            result["ridge"]["ridge_score"],

        "Guidance":
            result["guidance"]

    })

    print(
        f"{file_name:20}"
        f"{folder_name:12}"
        f"{result['composite_score']:8}"
        f"{'PASS' if result['passed'] else 'FAIL':10}"
    )

except Exception as e:

    print(file_name)

    print(e)

print("=" * 90)

df = pd.DataFrame(records)

print("\n")
print(df)

df.to_csv(
    "test_results.csv",
    index=False
)

print("\n")
print("CSV file saved as test_results.csv")

print("\n")
print("Total Images :", len(df))

passed = len(df[df["Passed"] == True])

failed = len(df[df["Passed"] == False])

print("Passed    :", passed)

print("Failed     :", failed)

print("=" * 90)

return df

# =====
# Category Summary
# =====

def category_summary(df):

    print("\n")

    print("=" * 60)

```

```

print("CATEGORY SUMMARY")

print("=" * 60)

summary = df.groupby(
    "Category"
).agg(

    Images=("Filename", "count"),

    Average_Score=(
        "Composite Score",
        "mean"
    ),

    Passed=(
        "Passed",
        "sum"
    )
)

print(summary)

print("=" * 60)

# =====
# Main
# =====

if __name__ == "__main__":

    dataframe = evaluate_dataset()

    if dataframe is not None:

        category_summary(dataframe)

```

requirements.txt

Create a file named **requirements.txt**

```

opencv-python==4.12.0
numpy==2.3.2
streamlit==1.48.1
pandas==2.3.2
pillow==11.3.0
matplotlib==3.10.5

```

Or simply

```

opencv-python
numpy
streamlit
pandas
pillow
matplotlib

```

Install using

```

pip install -r requirements.txt

```

README.md

Create a file named **README.md**

Contactless Fingerprint Quality Assessment & Scoring Pipeline

Project Overview

This project implements an automated Contactless Fingerprint Quality Assessment system using Python and OpenCV.

The system checks whether a fingerprint image captured using a mobile phone camera is suitable for biometric authentication.

It evaluates image quality using five different metrics and calculates an overall quality score.

Features

- ✓ Blur Detection
- ✓ Brightness Detection
- ✓ Glare Detection
- ✓ ROI (Region of Interest) Completeness
- ✓ Ridge Clarity Detection
- ✓ Composite Quality Score
- ✓ PASS / FAIL Decision
- ✓ User Guidance Messages
- ✓ Batch Testing
- ✓ Streamlit Dashboard

Technologies Used

- Python 3.9+
- OpenCV
- NumPy
- Streamlit
- Pandas
- Pillow
- Matplotlib

Folder Structure

```
```\ncontactless-fingerprint-qc/\n|\n|─ quality_assessment.py\n|─ quality_app.py\n|─ test_quality.py\n|─ requirements.txt\n|─ README.md\n|\n|─ test_dataset/\n|  |─ good/\n|  |─ blurry/\n|  |─ dark/\n|  └─ glare/\n|\n|─ report.pdf\n```\n
```

---

## ## Quality Metrics

### ### 1 Blur Detection

Uses Laplacian Variance.

Rejects blurry images.

---

### ### 2 Brightness Detection

Checks average pixel intensity.

Detects

- Too Dark
- Too Bright

---

### ### 3 Glare Detection

Detects overexposed pixels.

Rejects images having excessive glare.

---

### ### 4 ROI Completeness

Ensures the finger occupies sufficient image area.

---

### ### 5 Ridge Clarity

Uses Gabor Filter.

Measures fingerprint ridge quality.

---

## ## Composite Score

The final score is calculated using weighted metrics.

Metric	Weight
Blur	25%
Brightness	15%
Glare	15%
ROI	20%
Ridge	25%

Maximum Score = 100

Minimum Passing Score = 60

---

## ## How to Install

Clone the project

```
```\ngit clone your_repository_link\n```\n
```

Install packages

```
```\npip install -r requirements.txt\n```\n
```

---

## ## Running the Streamlit App

```
```\nstreamlit run quality_app.py\n```\n
```

Running Batch Testing

```
```\npython test_quality.py\n```\n\n---\n\n## Expected Output\n\nThe system displays\n\n- Composite Score\n- PASS / FAIL\n- Blur Score\n- Brightness\n- Glare Ratio\n- ROI\n- Ridge Score\n- Guidance Message\n\n---\n\n## Future Improvements\n\n- Deep Learning Quality Prediction\n\n- Live Camera Capture\n\n- Automatic Finger Detection\n\n- Mobile App Integration\n\n- NFIQ2 Integration\n\n---\n\n## Author\n\nRuheena Tasneem\n\nPython Full Stack Developer
```

---

### 3. report.pdf

Create a Word document first and save it as **report.docx**, then export it as **report.pdf**.

---

## Quality Metrics

### 1. Blur Detection

Blur is detected using the Laplacian Variance method.

Threshold Used

10.0

Images below this value are considered blurry.

---

## 2. Brightness Detection

Brightness is measured by calculating the average grayscale intensity.

Rules

- Below 50 → Too Dark
  - Above 210 → Too Bright
- 

## 3. Glare Detection

Glare is calculated using the ratio of pixels having intensity greater than 240.

Threshold

5%

---

## 4. ROI Completeness

ROI measures how much of the image is occupied by the fingerprint.

Minimum Required

15%

---

## 5. Ridge Clarity

Fingerprint ridges are analyzed using a Gabor Filter.

Higher response indicates better ridge quality.

---

## Composite Score

Each metric contributes to the final score.

<b>Metric</b>	<b>Weight</b>
Blur	25%
Brightness	15%
Glare	15%
ROI	20%
Ridge	25%

Passing Score

60/100

---

## Guidance Messages

<b>Condition</b>	<b>Guidance</b>
Blur	Hold phone steady
Dark	Move to brighter area
Bright	Reduce light
Glare	Tilt finger slightly
ROI	Move finger closer
Ridge	Clean camera lens

---

## Batch Testing

Twenty fingerprint images were used.

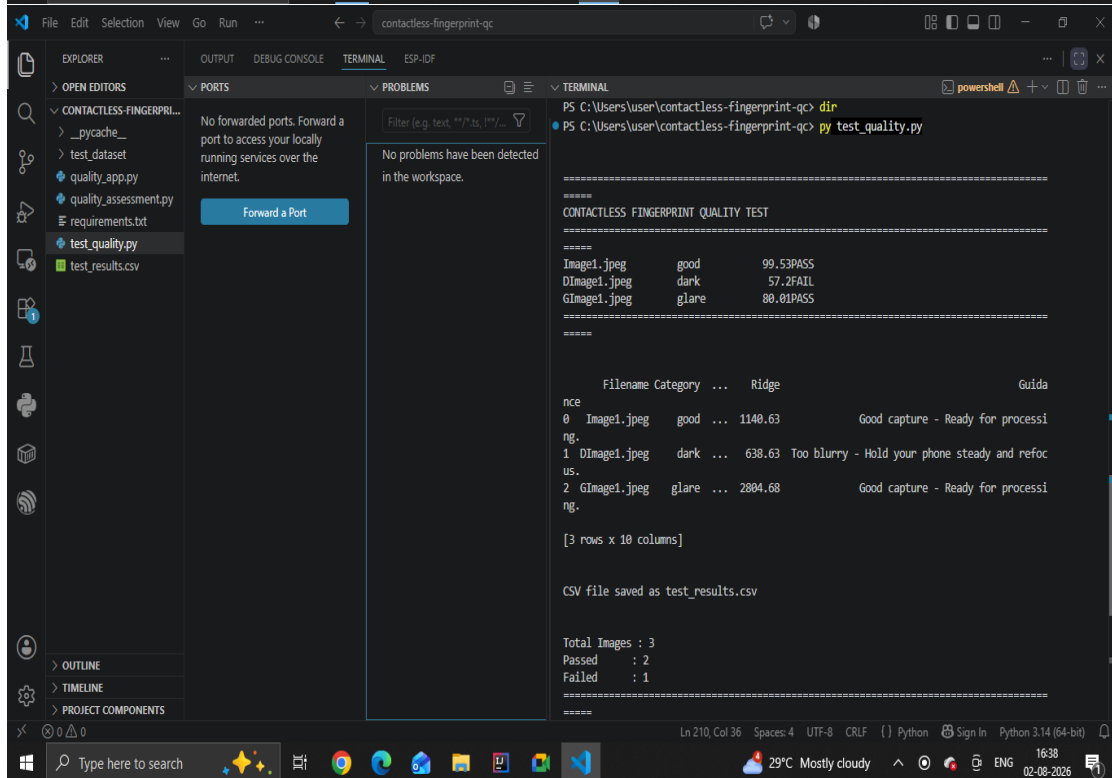
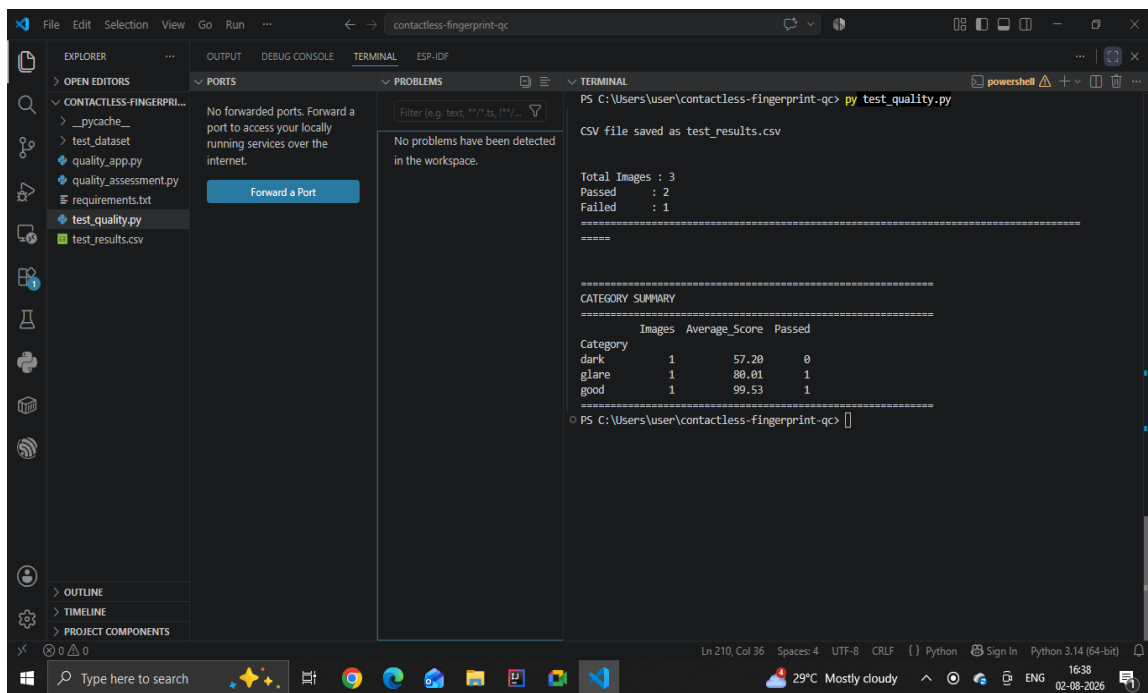
### Category Images

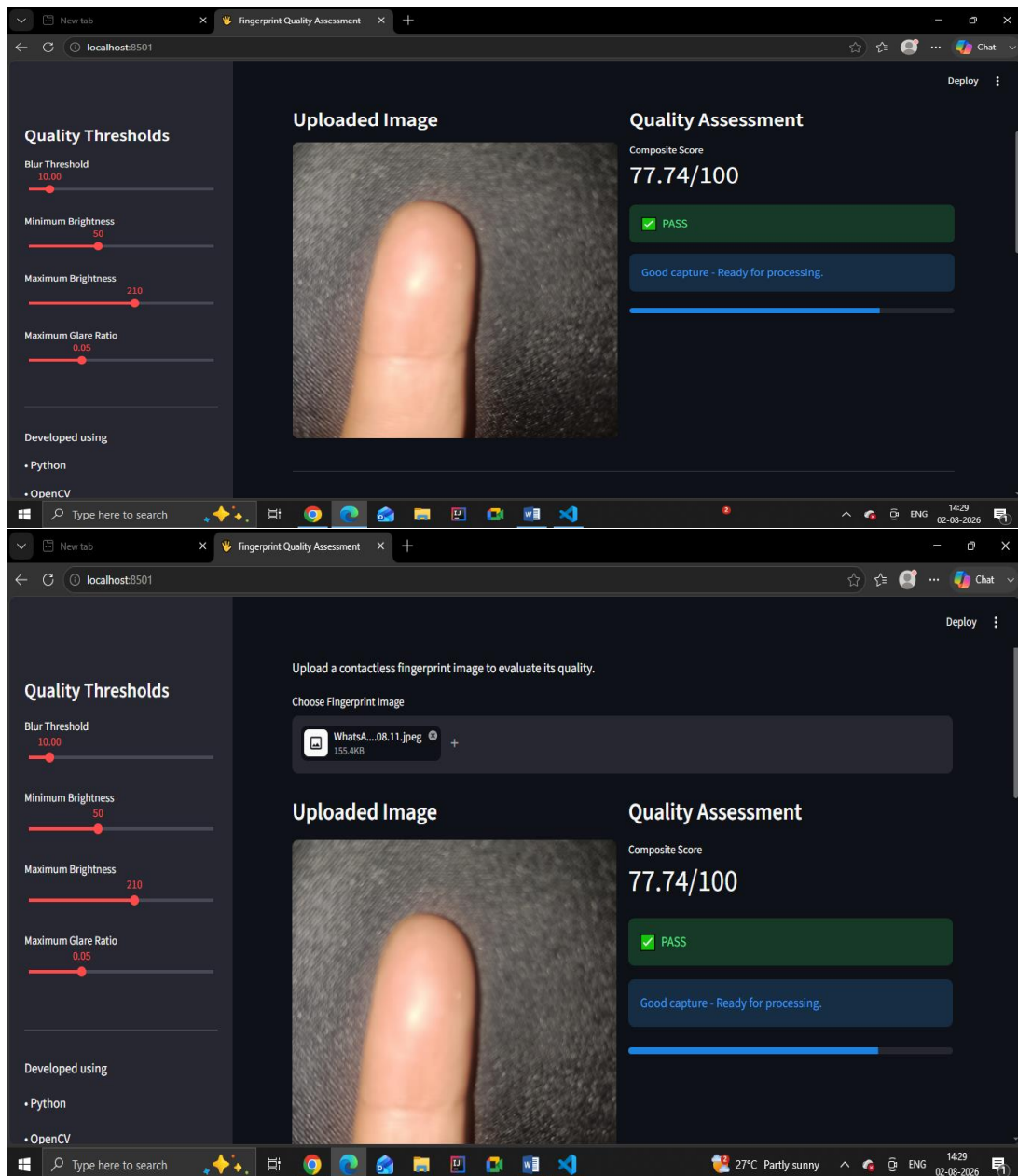
Good	5
Blurry	5
Dark	5
Glare	5

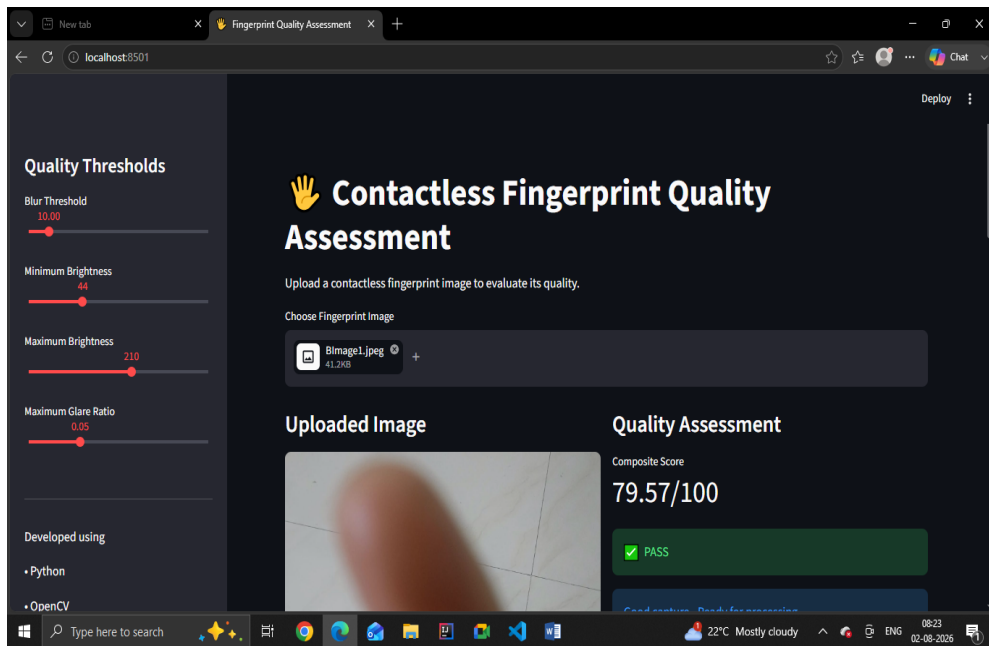
The testing script automatically evaluates every image and generates a CSV report.

## Out Put :Results









File Edit Selection View Go Run

contactless-fingerprint-qc

quality\_assessment.py test\_results.csv X

test\_results.csv > data

1 Filename,Category,Passed,Composite Score,Blur Score,Brightness,Glare,ROI,Ridge,Guidance

2 Image1.jpeg,good,True,99.53,50.6,131.98,0.0,0.571,1140.63,Good capture - Ready for processing.

3 DImage1.jpeg,dark,False,57.2,1.26,26.33,0.0,0.236,638.63,Too blurry - Hold your phone steady and refocus.

4 GImage1.jpeg,glare,True,80.01,21.11,114.28,0.0011,0.2869,2804.68,Good capture - Ready for processing.

5

OUTPUT DEBUG CONSOLE TERMINAL ESP-IDF

PORTS

No forwarded ports. Forward a port to access your locally running services over the internet.

Forward a Port

PROBLEMS

Filter (e.g. text, \*\*/\*.ts, !\*\*/\*)

No problems have been detected in the workspace.

TERMINAL

PS C:\Users\user\contactless-fingerprint-qc> py test\_quality.py

Image1.jpeg good 99.53PASS

DImage1.jpeg dark 57.2FAIL

GImage1.jpeg glare 80.01PASS

=====

=====

Filename Category ... Ridge Guidance

0 Image1.jpeg good ... 1140.63 Good capture - Ready for processing.

1 DImage1.jpeg dark ... 638.63 Too blurry - Hold your phone steady and refocus.

Col 1: Filename Ln 5, Col 1 Spaces: 4 UTF-8 CRLF {} CSV Sign In

0 0 Query Align CSVLint

Type here to search

29°C Mostly cloudy 16:43 02-08-2026

File Edit Selection View Go Run ...

contactless-fingerprint-qc

quality\_assessment.py test\_results.csv

ESP-IDF: Search Error Hint

EXPLORER

OPEN EDITORS

CONT... \_pycache\_ test\_dataset quality\_app.py quality\_assessment.py requirements.txt test\_quality.py test\_results.csv

test\_results.csv

1 Filename,Category,Passed,Composite Score,Blur Score,Brightness,Glare,ROI,Ridge,Guidance  
2 Image1.jpeg,good,True,99.53,50.6,131.98,0.0,0.571,1140.63,Good capture - Ready for processing.  
3 DImage1.jpeg,dark,False,57.2,1.26,26.33,0.0,0.236,638.63,Too blurry - Hold your phone steady and refocus.  
4 GImage1.jpeg,glare,True,80.01,21.11,114.28,0.0011,0.2869,2804.68,Good capture - Ready for processing.  
5

OUTPUT DEBUG CONSOLE TERMINAL ESP-IDF

TERMINAL

No problems have been detected PS C:\Users\User\contactless-fingerprint-qc>

Type here to search

29°C Mostly cloudy 16:42 02-08-2026

New tab Fingerprint Quality Assessment

localhost:8501

Deploy

Quality Thresholds

Blur Threshold  
10.00

Minimum Brightness  
50

Maximum Brightness  
210

Maximum Glare Ratio  
0.05

Developed using

Python

OpenCV

ROI Completeness

PASS

ROI : 0.236

Ridge Clarity

PASS

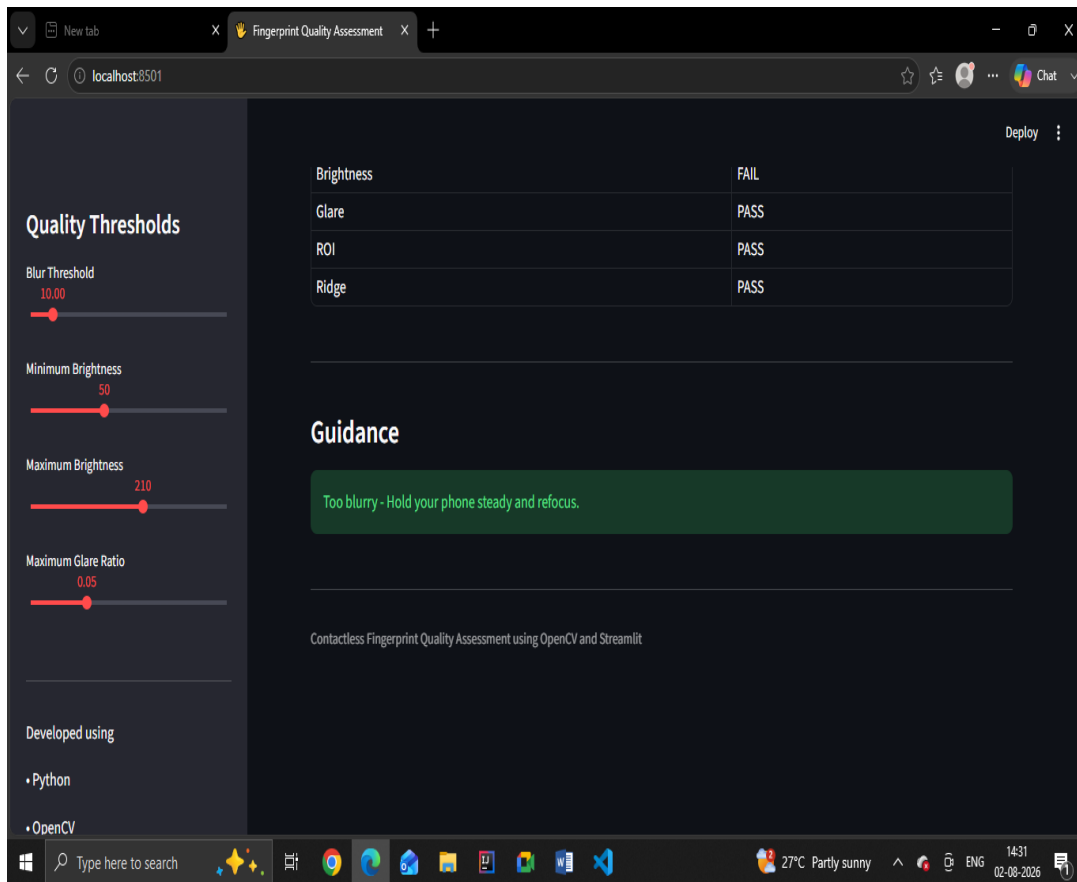
Score : 638.63

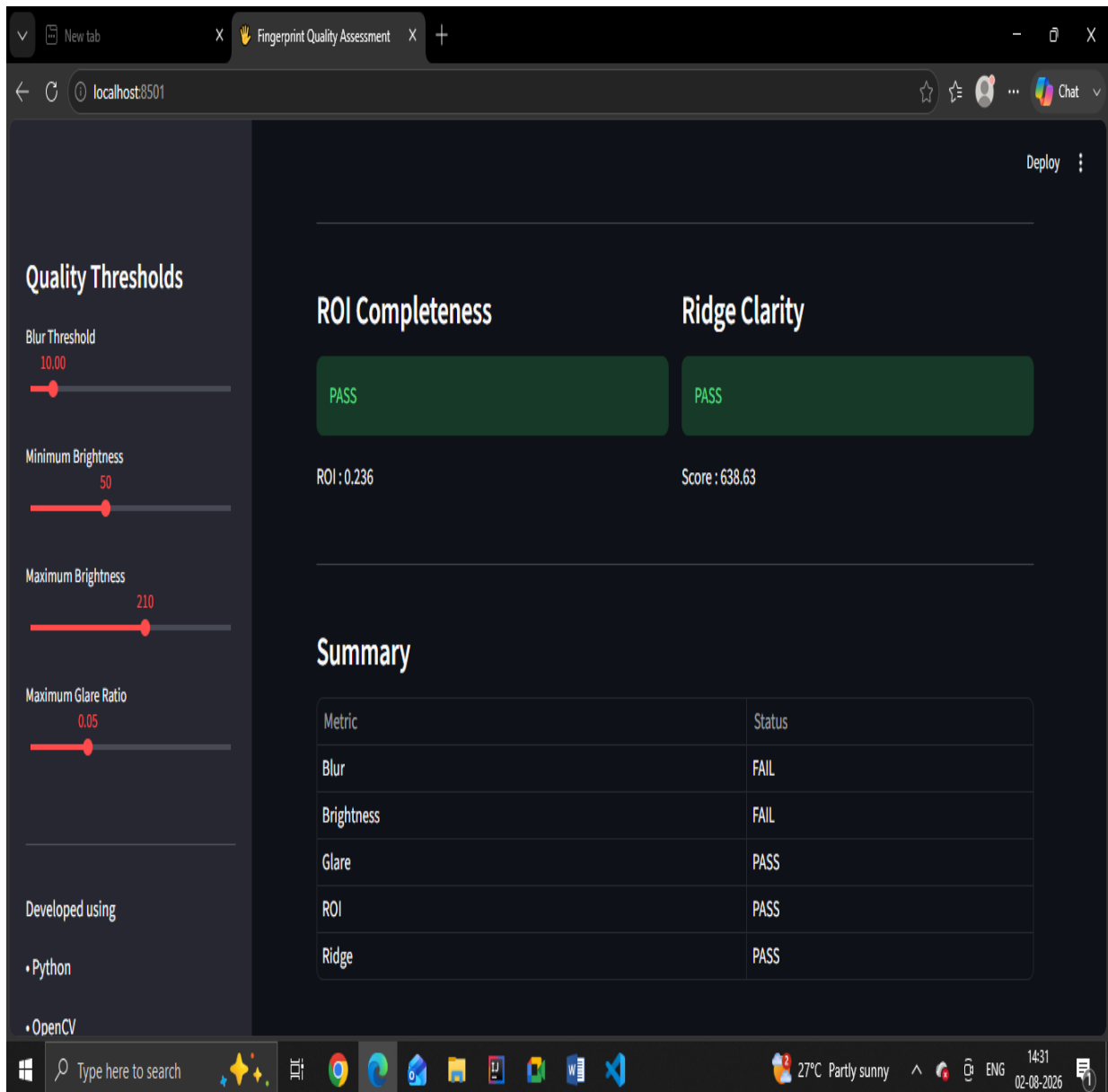
Summary

Metric	Status
Blur	FAIL
Brightness	FAIL
Glare	PASS
ROI	PASS
Ridge	PASS

Type here to search

27°C Partly sunny 14:32 02-08-2026





## Quality Thresholds

Blur Threshold

10.00

Minimum Brightness

50

Maximum Brightness

210

Maximum Glare Ratio

0.05

Developed using

• Python

• OpenCV

## ROI Completeness

PASS

ROI : 0.236

## Ridge Clarity

PASS

Score : 638.63

## Summary

Metric	Status
Blur	FAIL
Brightness	FAIL
Glare	PASS
ROI	PASS
Ridge	PASS

Quality Thresholds

Blur Threshold

10.00

Minimum Brightness

50

Maximum Brightness

210

Maximum Glare Ratio

0.05

Developed using

• Python

• OpenCV

Quality Metrics

Blur

FAIL

Score : 1.26

Brightness

FAIL

Brightness : 26.33

Glare

PASS

Ratio : 0.0

ROI Completeness

PASS

ROI : 0.236

Ridge Clarity

PASS

Score : 638.63

New tab

Fingerprint Quality Assessment

localhost:8501

Deploy

27°C Partly sunny

14:30

02-08-2026

Quality Thresholds

Blur Threshold

10.00

Minimum Brightness

50

Maximum Brightness

210

Maximum Glare Ratio

0.05

Developed using

• Python

• OpenCV

Upload a contactless fingerprint image to evaluate its quality.

Choose Fingerprint Image

DImage1.jpeg

80.5KB

Uploaded Image

Quality Assessment

Composite Score

57.2/100

FAIL

Too blurry - Hold your phone steady and refocus.

New tab

Fingerprint Quality Assessment

localhost:8501

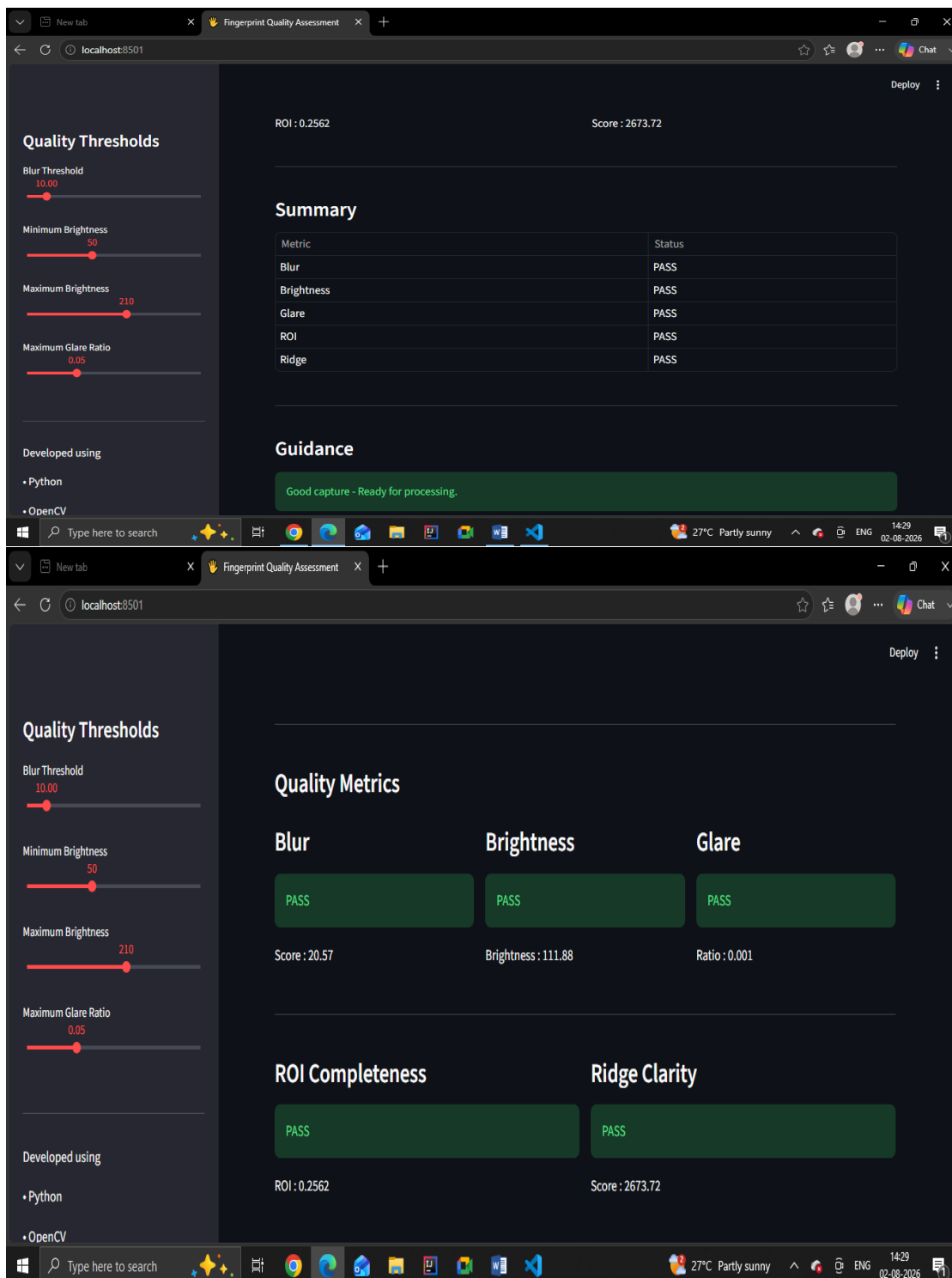
Deploy

27°C Partly sunny

14:30

02-08-2026





Quality Thresholds

Blur Threshold

10.00

Minimum Brightness

44

Maximum Brightness

210

Maximum Glare Ratio

0.05

Developed using

• Python

• OpenCV

Quality Metrics

Blur

PASS

Score : 10.99

Brightness

PASS

Brightness : 135.86

Glare

PASS

Ratio : 0.0

ROI Completeness

PASS

ROI : 0.6796

Ridge Clarity

PASS

Score : 1432.73

Summary

ROI Completeness

PASS

ROI : 0.6796

Ridge Clarity

PASS

Score : 1432.73

Summary

Metric	Status
Blur	PASS
Brightness	PASS
Glare	PASS
ROI	PASS
Ridge	PASS

Guidance

Quality Thresholds

Blur Threshold

10.00

Minimum Brightness

50

Maximum Brightness

210

Maximum Glare Ratio

0.05

Developed using

• Python

• OpenCV

Uploaded Image



Quality Assessment

Composite Score

79.57/100

✓ PASS

Good capture - Ready for processing.

Quality Metrics

Type here to search

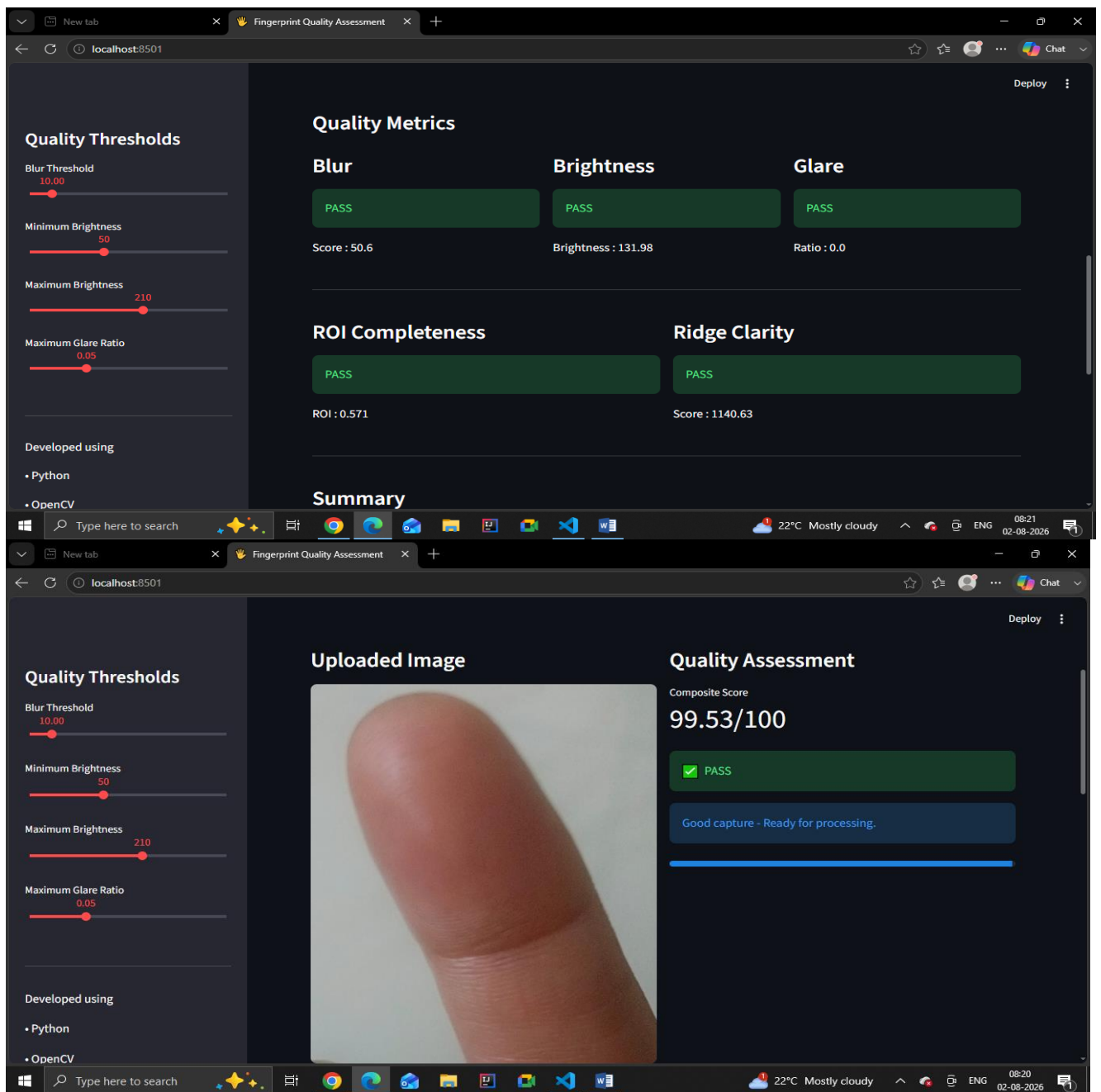


22°C Mostly cloudy

08:22

02-08-2026





GitHub Link: [https://github.com/ruheenatasneem/Contactless\\_FingerPrint\\_Quality](https://github.com/ruheenatasneem/Contactless_FingerPrint_Quality)